

Complete Table of Contents

Chapters 1–4

Randomized Reward Shaping Deep Reinforcement Learning
for Self-Adaptive Systems

Alireza Kavianifar

February 15, 2026

Contents

1.1 Background and Context

This section establishes the foundational context for self-adaptive systems and their role in managing runtime uncertainty.

1.1.1 Self-Adaptive Systems: motivation and examples

Self-adaptive systems autonomously adjust their behavior in response to changing environmental conditions and internal states. Examples include IoT networks, cloud computing infrastructures, and autonomous vehicles that must maintain quality-of-service goals despite unpredictable operating conditions.

1.1.2 The MAPE-K feedback loop and the Managed/Managing System abstraction

The MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) feedback loop provides a reference architecture for self-adaptive systems. The Managed System executes application logic, while the Managing System implements adaptation logic through continuous monitoring, analysis, planning, and execution cycles.

1.1.3 Limitations of rule-based and static adaptation

Traditional rule-based adaptation approaches struggle with:

- Large adaptation spaces where manual rule definition becomes intractable
- Dynamic environments where pre-defined rules cannot anticipate all scenarios
- Complex trade-offs between multiple quality-of-service objectives
- Inability to learn from experience and improve over time

1.2 Runtime Uncertainty in Self-Adaptive Systems

Uncertainty represents a fundamental challenge for adaptation decision-making, requiring systems to make effective choices despite incomplete or imperfect information.

1.2.1 Sources of uncertainty (environmental, system-level)

Uncertainty arises from multiple sources:

- **Environmental uncertainty:** Unpredictable interference, network conditions, workload variations
- **System-level uncertainty:** Hardware failures, software bugs, resource contention
- **Model uncertainty:** Imperfect models of system behavior and environment dynamics
- **Goal uncertainty:** Evolving or conflicting adaptation objectives

1.2.2 Impact of uncertainty on adaptation decisions (examples from DeltaIoT)

Using the DeltaIoT case study, we demonstrate how uncertainty affects adaptation outcomes. For example, network interference variations can cause the same configuration to produce different quality-of-service results, making it difficult to predict which adaptation option will achieve goals.

1.3 Learning-based Adaptation: Why Reinforcement Learning?

Reinforcement learning offers a principled framework for learning adaptation policies through trial-and-error interaction with uncertain environments.

1.3.1 RL benefits for runtime decision-making

Reinforcement learning provides:

- Automatic policy learning without manual rule engineering
- Ability to balance exploration and exploitation
- Handling of delayed rewards and long-term consequences
- Adaptation to changing environment dynamics

1.3.2 Limits of classic RL in large adaptation spaces

Classic RL approaches face challenges in self-adaptive systems:

- Sample inefficiency in large discrete action spaces
- Difficulty generalizing across heterogeneous environments
- Convergence to local optima
- High computational cost of retraining

1.4 What “Continuous” Means in This Thesis

Clarifying terminology is essential to avoid confusion with continuous-action reinforcement learning.

1.4.1 Continuous = continuous runtime learning / streaming adaptation cycles (not continuous-action control)

In this thesis, “continuous” refers to:

- **Continuous runtime learning:** Ongoing policy improvement through streaming adaptation cycles
- **Continuous operation:** Uninterrupted system operation during learning
- **NOT continuous-action spaces:** We use discrete adaptation options (DQN), not continuous control (PPO/DDPG)

1.5 Problem Statement (precise formulation extracted from the article)

Research Problem: How can self-adaptive systems effectively learn adaptation policies in large discrete adaptation spaces under runtime uncertainty, while maintaining continuous operation and achieving rapid convergence across heterogeneous environments?

The problem encompasses:

- Large adaptation spaces (hundreds to thousands of discrete options)
- Runtime uncertainty from environmental and system-level sources
- Need for sample-efficient learning
- Requirement for policy generalization across diverse conditions
- Constraint of continuous system operation during learning

1.6 Proposed Solution Overview: RS-DRL (high-level)

We propose Randomized Reward Shaping Deep Reinforcement Learning (RS-DRL), a novel approach that enhances DQN with selective reward reshaping to accelerate learning and improve generalization.

1.6.1 Key idea: randomized reward reshaping during experience replay

The core innovation is a randomized reward reshaping mechanism that:

- Selectively reshapes failed experiences during replay
- Treats certain failures as optimistic successes with probability ρ
- Accelerates learning by providing additional positive signals
- Promotes policy generalization across heterogeneous environments
- Operates during training without affecting runtime decision-making

1.6.2 Architectural placement within MAPE-K (on-demand retraining & async learning)

RS-DRL integrates with MAPE-K through:

- **Planning phase enhancement:** Learned policies guide adaptation option selection
- **Asynchronous learning:** Policy training occurs independently of runtime adaptation
- **On-demand retraining:** Triggered when environment changes are detected
- **Knowledge repository:** Stores trained models for runtime use

1.7 Research Objectives and Research Questions

Primary Objective: Develop and validate a learning-based approach for self-adaptive systems that achieves effective adaptation in large discrete spaces under uncertainty.

Research Questions:

1. How can reward shaping accelerate learning in large discrete adaptation spaces?
2. What mechanisms enable policy generalization across heterogeneous environments?
3. How does RS-DRL compare to baseline DQN and state-of-the-art approaches?
4. What architectural patterns support integration of learning with MAPE-K?

1.8 Contributions of the Thesis (expanded from the article’s contribution list)

This thesis makes the following contributions:

1. **Novel Algorithm:** RS-DRL with randomized reward reshaping mechanism
2. **Architectural Integration:** Process model for integrating RS-DRL with MAPE-K
3. **Empirical Validation:** Comprehensive evaluation on DeltaIoT case study
4. **Generalization Analysis:** Demonstration of cross-environment policy transfer
5. **Comparative Study:** Benchmarking against DQN, HER, and search-based methods

1.9 Research Methodology Overview

The research follows a systematic methodology combining theoretical modeling, algorithm design, and empirical validation.

1.9.1 Theoretical modeling and RL formulation

- Formalize self-adaptive system as Markov Decision Process
- Define state space, action space, and reward functions
- Establish theoretical foundations for reward shaping
- Analyze convergence properties

1.9.2 Algorithm design and reward shaping experiments

- Design RS-DRL algorithm with randomized reward reshaping
- Implement using Stable-Baselines3 DQN framework
- Conduct ablation studies on reshaping probability ρ
- Optimize hyperparameters through systematic experimentation

1.9.3 Case study (DeltaIoT): datasets, training regime, evaluation metrics

- Use DeltaIoT IoT network as motivating case study
- Train on datasets from Gheibi & Weyns with varying uncertainty
- Evaluate using metrics: cumulative reward, convergence speed, generalization
- Compare against baseline and state-of-the-art approaches

1.10 Thesis Organization (brief outline of subsequent chapters)

The remainder of this thesis is organized as follows:

- **Chapter 2:** Foundations and related work on self-adaptive systems, RL, and reward shaping
- **Chapter 3:** System model and problem formulation using DeltaIoT case study
- **Chapter 4:** Architecture and RS-DRL method with process model
- **Chapter 5:** Experimental evaluation and results analysis
- **Chapter 6:** Discussion, limitations, and future work
- **Chapter 7:** Conclusions and contributions summary

2.1 Self-Adaptive Systems: Concepts and Architectural Patterns

This section establishes the foundational concepts of self-adaptive systems and the MAPE-K reference architecture.

2.1.1 MAPE-K detailed (Monitor, Analyze, Plan, Execute, Knowledge)

The MAPE-K feedback loop consists of five components:

- **Monitor:** Collects runtime data from managed system and environment
- **Analyze:** Evaluates current state against adaptation goals
- **Plan:** Selects adaptation actions to achieve goals
- **Execute:** Applies selected adaptations to managed system
- **Knowledge:** Stores models, policies, and system state history

Each component has well-defined responsibilities and interfaces, enabling modular design and systematic reasoning about adaptation behavior.

2.1.2 Managed vs Managing System abstraction and temporal classification (monitoring vs asynchronous learning)

The Managed/Managing System abstraction separates:

- **Managed System:** Executes application logic, provides observability
- **Managing System:** Implements adaptation logic through MAPE-K

Temporal classification distinguishes:

- **Continuous processes:** Monitoring (always active)
- **Event-driven processes:** Analysis, Planning, Execution (triggered by conditions)
- **Asynchronous processes:** Learning (decoupled from runtime adaptation)

2.2 Uncertainty in Self-Adaptive Systems

Uncertainty is pervasive in self-adaptive systems and fundamentally affects adaptation decision-making.

2.2.1 Taxonomy of uncertainties (environmental, systemic, goal drift)

We adopt a taxonomy that categorizes uncertainty sources:

- **Environmental uncertainty:** External factors (interference, workload, user behavior)
- **Systemic uncertainty:** Internal factors (hardware variability, software bugs)
- **Model uncertainty:** Imperfect models of system and environment
- **Goal uncertainty:** Evolving or conflicting objectives

2.2.2 Effects of uncertainty on verification and adaptation (motivating evidence from DeltaIoT)

Using DeltaIoT as evidence, we demonstrate:

- Same configuration produces different QoS under varying interference
- Verification results show probability distributions, not deterministic outcomes
- Adaptation decisions must account for uncertainty to avoid goal violations
- Learning-based approaches can adapt to uncertainty through experience

Figure 2 from the article shows how packet loss varies across interference levels, motivating the need for uncertainty-aware adaptation.

2.3 Reinforcement Learning Foundations for SAS

This section establishes RL foundations relevant to self-adaptive systems.

2.3.1 MDPs, policy/value formulation and why RL suits adaptation problems

Markov Decision Process (MDP) formulation:

- State space $\mathcal{S}$: System and environment observations
- Action space $\mathcal{A}$: Available adaptation options
- Transition function $P(s'|s, a)$: State dynamics
- Reward function $R(s, a, s')$: Quality-of-service feedback
- Discount factor γ : Future reward weighting

Policy and value functions:

- Policy $\pi(a|s)$: Probability of selecting action a in state s
- State-value $V^\pi(s)$: Expected return from state s under policy π
- Action-value $Q^\pi(s, a)$: Expected return from taking action a in state s

RL suits adaptation because it:

- Handles sequential decision-making under uncertainty
- Learns from experience without requiring complete system models
- Balances immediate and long-term objectives through discount factor
- Adapts to changing environment dynamics

2.3.2 Known challenges: local optima, generalization, retraining cost

Classic RL faces challenges in self-adaptive systems:

- **Local optima:** Greedy policies may converge to suboptimal solutions
- **Sample inefficiency:** Large action spaces require many experiences
- **Generalization:** Policies may not transfer across environments
- **Retraining cost:** Computational overhead of policy updates
- **Exploration-exploitation:** Balancing learning and performance

2.4 Deep Reinforcement Learning in Large Discrete Adaptation Spaces

Deep RL extends classic RL to handle large state and action spaces through function approximation.

2.4.1 DQN architecture and rationale for discrete adaptation options (why DQN was chosen in the article)

Deep Q-Network (DQN) architecture:

- Neural network approximates $Q(s, a)$ function
- Input: State representation $s \in \mathbb{R}^d$
- Output: Q-values for all actions $[Q(s, a_1), \dots, Q(s, a_n)]$
- Loss: Temporal difference error minimization

Why DQN for self-adaptive systems:

- Adaptation spaces are naturally discrete (configuration options)
- DQN handles large discrete action spaces efficiently
- Single forward pass evaluates all options simultaneously
- Proven stability and convergence properties

2.4.2 Experience replay, target networks, stability techniques used

DQN employs several techniques for stable learning:

- **Experience replay:** Store experiences (s, a, r, s') in memory, sample randomly for training
- **Target network:** Separate network for computing TD targets, updated periodically
- **ϵ -greedy exploration:** Balance exploration and exploitation
- **Gradient clipping:** Prevent exploding gradients
- **Reward normalization:** Stabilize learning across different scales

2.5 Reward Shaping and Experience Relabeling (state of the art)

Reward shaping techniques modify reward signals to accelerate learning and improve generalization.

2.5.1 HER and goal-conditioned methods — contrast to RS-DRL (no pre-defined goals)

Hindsight Experience Replay (HER):

- Relabels failed experiences with achieved goals
- Requires explicit goal representation in state space
- Effective for sparse reward environments
- Assumes goals can be extracted from final states

Contrast with RS-DRL:

- RS-DRL does not require pre-defined goal representations
- Reshaping based on reward thresholds, not goal achievement
- Applicable to multi-objective optimization without explicit goals
- Randomized reshaping provides stochastic exploration benefit

2.5.2 Other generalization techniques

Additional techniques for improving RL generalization:

- **Domain randomization:** Train on diverse environment variations
- **Meta-learning:** Learn to adapt quickly to new environments
- **Transfer learning:** Initialize policies from related tasks
- **Curriculum learning:** Gradually increase task difficulty

2.6 Related Work: classification and gaps

This section surveys existing approaches to learning-based adaptation and identifies research gaps.

2.6.1 ML-based methods, search-based planners, RL-based adaptation — strengths and weaknesses (article’s survey)

Machine Learning-based methods:

- Supervised learning for adaptation prediction
- Strengths: Fast inference, interpretable models
- Weaknesses: Require labeled training data, limited to seen scenarios

Search-based planners:

- Optimization algorithms (genetic algorithms, simulated annealing)
- Strengths: No training required, handle complex objectives
- Weaknesses: Computational overhead, may not scale to large spaces

RL-based adaptation:

- Learn policies through trial-and-error
- Strengths: Adapt to uncertainty, improve over time
- Weaknesses: Sample inefficiency, generalization challenges

2.6.2 Identified gap: scalable, uncertainty-aware, continuous runtime DRL for large adaptation spaces

Research gap: Existing approaches lack:

- Scalable learning in large discrete adaptation spaces (100s-1000s options)
- Effective generalization across heterogeneous environments
- Sample-efficient learning under runtime uncertainty
- Continuous operation during learning (asynchronous training)
- Principled reward shaping for multi-objective adaptation

RS-DRL addresses this gap through randomized reward reshaping and asynchronous learning architecture.

2.7 Chapter Summary (lead into Chapter 3's formal modeling)

This chapter established foundational concepts:

- Self-adaptive systems and MAPE-K architecture
- Uncertainty taxonomy and impact on adaptation
- Reinforcement learning formulation and DQN architecture
- Reward shaping techniques and related work
- Research gap motivating RS-DRL approach

Chapter 3 formalizes the adaptation problem using the DeltaIoT case study, providing concrete MDP formulation and experimental setup.

3.1 Motivating Case Study: DeltaIoT System (detailed description)

DeltaIoT serves as the motivating case study, providing a concrete context for evaluating RS-DRL.

3.1.1 Network, motes, links, configuration parameters and examples

DeltaIoT system architecture:

- IoT sensor network with multiple motes (sensor nodes)
- Wireless communication links subject to interference
- Gateway collecting sensor data
- Quality-of-service requirements: packet loss, latency, energy consumption

Configuration parameters:

- Transmission power levels (affects range and energy)
- Distribution strategy (single-path vs multi-path routing)
- Packet size (affects transmission time and reliability)

Example configurations:

- Low power + single-path: Energy-efficient but vulnerable to interference
- High power + multi-path: Reliable but energy-intensive
- Medium power + adaptive routing: Balanced trade-off

3.2 Modeling Uncertainty in DeltaIoT and SAS

Uncertainty in DeltaIoT arises from environmental interference and message load variability.

3.2.1 Interference/noise ranges, message-load variability, data shifts (Fig.2 evidence)

Interference uncertainty:

- Interference levels vary from 0.0 to 1.0 (normalized)
- Affects packet loss probability and transmission reliability
- Figure 2 shows packet loss increasing with interference
- Same configuration produces different outcomes under different interference

Message load variability:

- Number of messages varies over time
- Affects network congestion and latency
- Creates temporal dependencies in adaptation decisions

Data shifts:

- Training and deployment environments may differ
- Requires policy generalization across interference ranges
- Motivates need for robust learning approaches

3.2.2 How uncertainty affects QoS metrics (packet loss, latency, energy)

Packet loss:

- Increases with interference and low transmission power
- Probabilistic: same configuration yields different loss rates
- Critical metric for reliability requirements

Latency:

- Affected by retransmissions due to packet loss
- Increases with network congestion
- Important for time-sensitive applications

Energy consumption:

- Increases with transmission power and retransmissions
- Trade-off with reliability and latency
- Critical for battery-powered sensor nodes

3.3 Adaptation Space and Configuration Representation

The adaptation space defines the set of available configuration options.

3.3.1 Definition: adaptation option, parameter encoding, scale (216 / 1,296 / 4,096)

Adaptation option: A complete system configuration specifying all parameter values.

Parameter encoding:

- Transmission power: 3 levels (low, medium, high)
- Distribution strategy: 2 options (single-path, multi-path)
- Packet size: 3 sizes (small, medium, large)
- Additional parameters depending on experiment setup

Adaptation space scale:

- Small: 216 discrete options
- Medium: 1,296 discrete options
- Large: 4,096 discrete options

These scales represent realistic adaptation spaces in self-adaptive systems, challenging for both search-based and learning-based approaches.

3.3.2 Implications for search/learning complexity

Search complexity:

- Exhaustive search requires evaluating all options
- Verification cost per option: 100s of simulations
- Total search time becomes prohibitive in large spaces

Learning complexity:

- Large action spaces require more training experiences
- Risk of overfitting to training environments
- Need for sample-efficient learning algorithms

3.4 RL Formalization for the Adaptive Problem (exact equations from paper)

This section provides the formal MDP formulation for DeltaIoT adaptation.

3.4.1 State space definition (e.g., $S = (E, P, L)$)

State representation: $s_t = (E_t, P_t, L_t)$ where:

- E_t : Energy consumption level
- P_t : Packet loss rate
- L_t : Latency measure

Additional state features may include:

- Current configuration parameters
- Recent trend indicators
- Environmental observations (interference estimates)

State space dimensionality: $s \in \mathbb{R}^d$ where d depends on feature engineering.

3.4.2 Action space: discrete adaptation options and encoding

Action space: $\mathcal{A} = \{a_1, a_2, \dots, a_n\}$ where $n \in \{216, 1296, 4096\}$

Action encoding:

- Each action a_i corresponds to a unique configuration
- Integer encoding: $a_i \in \{0, 1, \dots, n-1\}$
- One-hot encoding for neural network input (if needed)

3.4.3 Reward functions: multi-objective normalization, threshold/setpoint formulations (include equations 11–12 and Table 2)

Multi-objective reward function:

$$r_t = w_E \cdot r_E(s_t) + w_P \cdot r_P(s_t) + w_L \cdot r_L(s_t) \quad (1)$$

where $w_E + w_P + w_L = 1$ are objective weights.

Threshold-based formulation (Equation 11 from article):

$$r_i(s_t) = \begin{cases} 1 & \text{if } m_i(s_t) \leq \theta_i \\ -1 & \text{otherwise} \end{cases} \quad (2)$$

where $m_i(s_t)$ is metric i value and θ_i is threshold.

Setpoint-based formulation (Equation 12 from article):

$$r_i(s_t) = 1 - \frac{|m_i(s_t) - \theta_i|}{\theta_i} \quad (3)$$

Table 2: Reward function parameters

Metric	Threshold θ_i	Weight w_i
Energy	0.5	0.33
Packet Loss	0.1	0.34
Latency	100 ms	0.33

Table 1: Reward function parameters for DeltaIoT experiments.

3.4.4 Transition dynamics (how verifier / ActivFORMS produces next states)

Transition function: $s_{t+1} \sim P(s'|s_t, a_t)$

Verification-based transitions:

- UPPAAL-SMC statistical model checker simulates system behavior
- ActivFORMS execution engine provides runtime verification
- Multiple simulation runs produce probability distributions
- Next state sampled from distribution: $s_{t+1} \sim P(\cdot|s_t, a_t)$

Stochastic transitions:

- Same (s_t, a_t) pair can produce different s_{t+1}
- Captures uncertainty in system behavior
- Requires learning robust policies

3.5 Continuous / Streaming Training Assumptions

This section clarifies the continuous learning paradigm used in RS-DRL.

3.5.1 Treating adaptation cycles as a continuous stream (training horizons and validation splits per instance)

Continuous stream paradigm:

- Adaptation cycles arrive continuously during operation
- Each cycle generates experience tuple (s_t, a_t, r_t, s_{t+1})
- Experiences stored in replay memory for training
- No episodic boundaries in runtime operation

Training horizons:

- Training occurs over fixed time horizons (e.g., 1000 cycles)
- Validation performed on separate environment instances
- Cross-validation across interference levels

3.5.2 Temporal separation: runtime adaptation vs asynchronous learning (design assumption)

Key architectural assumption: Runtime adaptation and learning are temporally separated.

Runtime adaptation:

- Uses pre-trained model for inference
- Pure forward pass through neural network
- No gradient computation or weight updates
- Fast, deterministic decision-making

Asynchronous learning:

- Occurs in background process
- Samples experiences from replay memory
- Updates model weights through gradient descent
- Stores updated model for future runtime use

This separation ensures runtime stability while enabling continuous improvement.

3.6 Verification & Runtime Models for Option Analysis

Verification provides quality estimates for adaptation options under uncertainty.

3.6.1 UPPAAL-SMC models and ActivFORMS execution engine (role in reward calculation)

UPPAAL-SMC:

- Statistical model checker for stochastic timed automata
- Simulates system behavior under uncertainty
- Produces probability estimates for property satisfaction
- Used during training to evaluate adaptation options

ActivFORMS:

- Runtime verification engine
- Executes formal models with current state parameters
- Provides quality-of-service predictions
- Generates reward signals for learning

Role in reward calculation:

1. Agent selects action a_t in state s_t
2. Verifier evaluates (s_t, a_t) using formal models
3. Simulation produces QoS metrics: (E, P, L)
4. Reward computed from metrics: $r_t = f(E, P, L)$
5. Next state extracted from simulation: s_{t+1}

3.7 Formal Problem Statement (thesis version) — precise objective and constraints (ready to feed Chapter 4)

Formal Problem Statement:

Given:

- Self-adaptive system with discrete adaptation space $\mathcal{A}$, $|\mathcal{A}| \in [100, 10000]$
- State space $\mathcal{S} \subset \mathbb{R}^d$ representing system and environment
- Stochastic transition dynamics $P(s'|s, a)$ with runtime uncertainty
- Multi-objective reward function $R(s, a, s')$ encoding QoS goals
- Heterogeneous environment instances $\mathcal{E} = \{e_1, \dots, e_k\}$

Find:

- Policy $\pi^* : \mathcal{S} \rightarrow \mathcal{A}$ that maximizes expected cumulative reward:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi, e \sim \mathcal{E}} \left[\sum_{t=0}^T \gamma^t R(s_t, a_t, s_{t+1}) \right] \quad (4)$$

Subject to constraints:

- Sample efficiency: Learn with limited training experiences
- Generalization: Policy performs well across all $e \in \mathcal{E}$
- Continuous operation: System remains operational during learning
- Computational feasibility: Training and inference within resource bounds

This formulation guides the RS-DRL architecture and algorithm design in Chapter 4.

4.1 Chapter overview & architectural perspective (scope, process-oriented organization)

This chapter presents the RS-DRL architecture and method, organized by process flow rather than component structure.

Chapter scope:

- System-level architecture integrating RS-DRL with MAPE-K
- Nine-stage adaptation cycle from monitoring to execution
- Asynchronous learning process with reward reshaping
- Knowledge management and process coordination

Process-oriented organization:

- Each section answers: “What happens next, and why?”
- MAPE-K serves as explanatory lens, not primary structure
- Decomposed figures aligned with section boundaries
- Complete workflows provided in appendices

4.2 System-level Self-Adaptation Loop (nine-stage continuous adaptation cycle)

The RS-DRL-based self-adaptive system operates through a nine-stage continuous cycle.

4.2.1 Stage 1: Runtime data collection (sensors → Monitor)

Process: Monitor collects runtime data from managed system and environment.

Data sources:

- System metrics: energy consumption, packet loss, latency
- Environment observations: interference estimates, workload
- Configuration state: current parameter values

Output: Raw sensor data ready for preprocessing.

4.2.2 Stage 2: State forwarding to RS-DRL module (experience collection)

Process: Validated state vector forwarded to RS-DRL module for experience collection.

Experience tuple construction:

- Previous state s_t
- Action taken a_t
- Reward received r_t
- Current state s_{t+1}

Storage: Experience (s_t, a_t, r_t, s_{t+1}) stored in replay memory.

4.2.3 Stage 3: Asynchronous model training and storage (replay $\rightarrow$ train $\rightarrow$ store)

Process: Background learning process trains model using replay memory.

Training steps:

1. Sample minibatch from replay memory
2. Apply randomized reward reshaping
3. Compute TD targets and loss
4. Update Q-network weights via gradient descent
5. Store updated model in Knowledge repository

Asynchronous execution: Training occurs independently of runtime adaptation.

4.2.4 Stage 4: State forwarding to Analyzer

Process: State vector forwarded to Analyzer for goal evaluation.

Analyzer input:

- Current state s_t
- Adaptation goals and thresholds
- Historical state trajectory

4.2.5 Stage 5: Adaptation triggering (Analyzer detects violations)

Process: Analyzer evaluates whether adaptation is needed.

Triggering conditions:

- QoS metric exceeds threshold
- Trend indicates future violation
- Environment change detected
- Periodic adaptation schedule

Output: Adaptation trigger signal sent to Planner.

4.2.6 Stage 6: Trained model retrieval

Process: Planner retrieves latest trained model from Knowledge repository.

Model selection:

- Latest successfully trained model
- Model validated on current environment type
- Fallback to previous model if training failed

4.2.7 Stage 7: Adaptation option prediction (Adaptation Option Predictor)

Process: Planner uses model to predict best adaptation option.

Prediction steps:

1. Forward pass: $Q(s_t, \cdot) = \text{model}(s_t)$
2. Compute Q-values for all actions
3. Select action: $a_t = \arg \max_a Q(s_t, a)$ (greedy) or ϵ -greedy

Output: Predicted adaptation option a_t .

4.2.8 Stage 8: Adaptation option retrieval & verification

Process: Executor retrieves detailed specification and verifies option.

Verification:

- UPPAAL-SMC simulates (s_t, a_t) configuration
- Estimates QoS metrics and goal satisfaction probability
- Provides decision justification

Output: Verified adaptation specification ready for execution.

4.2.9 Stage 9: Adaptation application (Executor)

Process: Executor applies adaptation to managed system.

Execution steps:

1. Translate adaptation option to system commands
2. Apply configuration changes to managed system
3. Verify successful reconfiguration
4. Log adaptation event

Loop closure: Monitoring resumes (Stage 1), closing the adaptation loop.

4.3 Mapping RS-DRL to MAPE-K (conceptual mapping and roles)

This section provides conceptual mapping of RS-DRL stages to MAPE-K components.

MAPE-K mapping:

- **Monitor** (Stages 1, 2, 4): Data collection, state construction, dissemination
- **Analyze** (Stages 4, 5): Goal evaluation, adaptation triggering
- **Plan** (Stages 5, 6, 7): Model retrieval, option prediction
- **Execute** (Stages 8, 9): Verification, adaptation application
- **Knowledge** (Stages 3, 6, 8): Model storage, option repository, state history

RS-DRL role: Enhances Planning phase with learned intelligence, operates asynchronously in Knowledge component.

4.4 Runtime Monitoring and State Construction

Monitoring transforms raw sensor data into validated state representations.

4.4.1 Data collection, preprocessing, feature extraction, state vector construction

Data collection:

- Sensors provide raw measurements
- Sampling rate: continuous or periodic
- Data buffering for temporal features

Preprocessing:

- Filtering: Remove noise and outliers

- Normalization: Scale features to $[0, 1]$ or standardize
- Aggregation: Compute moving averages, trends

Feature extraction:

- QoS metrics: energy, packet loss, latency
- Derived features: rates of change, violation indicators
- Context features: current configuration, time of day

State vector construction:

- Assemble features into vector $s_t \in \mathbb{R}^d$
- Validate: Check for missing values, range violations
- Disseminate: Forward to Analyzer and RS-DRL module

4.5 Analysis and Adaptation Triggering

Analysis evaluates system state against adaptation goals and triggers planning when needed.

4.5.1 Change detection & thresholding assumptions (when retraining is triggered)

Goal evaluation:

- Compare metrics against thresholds: $m_i(s_t) \leq \theta_i$
- Evaluate multi-objective satisfaction
- Detect trends indicating future violations

Adaptation triggering:

- Immediate: Threshold violation detected
- Predictive: Trend analysis predicts violation
- Periodic: Time-based adaptation schedule

Retraining triggers:

- Environment change detected (interference shift)
- Policy performance degradation
- Sufficient new experiences collected
- Scheduled retraining interval

4.6 Planning & Adaptation Option Prediction

Planning uses the trained RS-DRL model to select adaptation options.

4.6.1 Adaptation Option Selector (ϵ -greedy + DQN) & architecture of predictor

Adaptation Option Predictor architecture:

- Input layer: State vector $s_t \in \mathbb{R}^d$
- Hidden layers: Fully connected neural network
- Output layer: Q-values for all actions $Q(s_t, a_i)$, $i = 1, \dots, n$

Action selection:

- Training: ϵ -greedy exploration

$$a_t = \begin{cases} \text{random action} & \text{with probability } \epsilon \\ \arg \max_a Q(s_t, a) & \text{with probability } 1 - \epsilon \end{cases} \quad (5)$$

- Runtime: Greedy selection ($\epsilon = 0$)

$$a_t = \arg \max_a Q(s_t, a) \quad (6)$$

Pure inference at runtime: No learning, no replay sampling, no weight updates.

4.7 Verification of Adaptation Options

Verification provides quality estimates and decision justification for selected options.

4.7.1 Runtime model checking with UPPAAL-SMC & how verifier informs reward

Verification process:

1. Parameterize runtime model with (s_t, a_t)
2. Execute UPPAAL-SMC statistical model checking
3. Run multiple simulations (e.g., 1000 runs)
4. Collect QoS metrics from simulations
5. Compute quality estimates and probabilities

Dual role of verification:

- **Runtime:** Provides decision justification and confidence estimates
- **Learning:** Generates reward signals r_t from QoS metrics

Reward generation:

$$r_t = f(\text{QoS metrics from verification}) \quad (7)$$

where f is the multi-objective reward function defined in Section 3.4.3.

4.8 Execution and Adaptation Application

Execution translates selected adaptation options into system reconfigurations.

4.8.1 Executor behavior and coordination with Knowledge repository

Adaptation specification retrieval:

- Query Knowledge repository with action ID a_t
- Retrieve complete configuration specification
- Includes all parameter values and dependencies

Execution process:

1. Validate adaptation specification
2. Translate to system-specific commands
3. Apply configuration changes atomically
4. Verify successful reconfiguration
5. Update system state in Knowledge

Error handling:

- Rollback on execution failure
- Fallback to safe default configuration
- Log error for analysis

4.9 Asynchronous RS-DRL Learning and Policy Update (core algorithmic chapter)

This section presents the core RS-DRL algorithm with randomized reward reshaping.

4.9.1 Overall RS-DRL algorithm (Algorithm 1) and parameters (η , γ , ϵ , ρ)

Algorithm 1: RS-DRL Training

Parameters:

- η : Learning rate
- γ : Discount factor
- ϵ : Exploration rate
- ρ : Reward reshaping probability

Algorithm outline:

1. Initialize Q-network $Q(s, a; \theta)$ and target network $Q(s, a; \theta^-)$
2. Initialize replay memory $\mathcal{D}$
3. For each training episode:
 - (a) Observe initial state s_0
 - (b) For each time step t :
 - i. Select action a_t using ϵ -greedy policy
 - ii. Execute action, observe reward r_t and next state s_{t+1}
 - iii. Store experience (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$
 - iv. Sample minibatch from $\mathcal{D}$
 - v. Apply randomized reward reshaping (Algorithm 2)
 - vi. Compute TD targets and update Q-network
 - (c) Periodically update target network: $\theta^- \leftarrow \theta$
4. Store trained model in Knowledge repository

4.9.2 Replay Memory & randomized reward reshaping (Algorithm 2) — rationale & implementation

Algorithm 2: Randomized Reward Reshaping

Input: Minibatch of experiences $\{(s_i, a_i, r_i, s'_i)\}$

Output: Reshaped minibatch $\{(s_i, a_i, \tilde{r}_i, s'_i)\}$

Algorithm:

1. For each experience (s_i, a_i, r_i, s'_i) in minibatch:
 - (a) If $r_i < 0$ (failed experience):
 - i. With probability ρ : $\tilde{r}_i = |r_i|$ (reshape to positive)
 - ii. With probability $1 - \rho$: $\tilde{r}_i = r_i$ (keep original)
 - (b) Else: $\tilde{r}_i = r_i$ (keep positive rewards unchanged)
2. Return reshaped minibatch

Rationale:

- Treats certain failures as optimistic successes
- Provides additional positive learning signals
- Accelerates exploration of promising regions
- Promotes generalization across environments
- Randomization prevents overfitting to reshaped distribution

Implementation details:

- Reshaping applied during training only, not at runtime
- Probability ρ is hyperparameter (typically 0.1-0.3)
- Original experiences preserved in replay memory
- Reshaping creates temporary training signal

4.9.3 Model training, target networks, loss function, stability practices

TD target computation:

$$y_i = r_i + \gamma \max_{a'} Q(s'_i, a'; \theta^-) \quad (8)$$

Loss function:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(y - Q(s, a; \theta))^2] \quad (9)$$

Gradient update:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta) \quad (10)$$

Target network update:

- Periodic hard update: $\theta^- \leftarrow \theta$ every C steps
- Or soft update: $\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$

Stability practices:

- Gradient clipping: Prevent exploding gradients
- Reward normalization: Stabilize learning across scales
- Experience replay: Break temporal correlations
- Target network: Stabilize TD targets
- Learning rate scheduling: Reduce η over time

4.10 Knowledge Management and Process Coordination

Knowledge serves as shared memory coordinating heterogeneous processes.

4.10.1 Knowledge repository design (models, adaptation options, state history)

Knowledge components:

- **Model repository:** Trained Q-networks, metadata, versioning
- **Adaptation option catalog:** Configuration specifications, parameters
- **State history:** Time-series of system states and adaptations
- **Replay memory:** Experience tuples for training

Data structures:

- **Models:** Serialized neural network weights, architecture specs
- **Options:** JSON/XML configuration files
- **History:** Time-series database or log files
- **Replay:** Circular buffer or database

4.10.2 Read/write coordination contracts and heterogeneous process synchronization

Read/write contracts:

- **Monitor writes:** State vectors (Stage 1)
- **Learning writes:** Trained models (Stage 3)
- **Planner reads:** Trained models (Stage 6)
- **Executor reads:** Adaptation specifications (Stage 8)

Synchronization mechanisms:

- **Atomic writes:** Ensure model updates are complete
- **Versioning:** Track model versions, enable rollback
- **Locks:** Prevent concurrent write conflicts
- **Notifications:** Signal when new models available

Temporal decoupling:

- Runtime processes read latest available model
- Learning processes write asynchronously
- No blocking between runtime and learning

4.11 Implementation details & hyperparameter choices

This section provides practical implementation details for reproducibility.

4.11.1 Gymnasium environment, Stable-Baselines3 DQN extension, dataset traces (Gheibi & Weyns)

Gymnasium environment:

- Custom environment wrapping DeltaIoT simulator
- State space: $\mathbb{R}^d$ (continuous)
- Action space: Discrete(n) where $n \in \{216, 1296, 4096\}$
- Reward: Multi-objective function from Section 3.4.3
- Episode termination: Fixed horizon or goal achievement

Stable-Baselines3 DQN extension:

- Base: SB3 DQN implementation
- Extension: Custom callback for reward reshaping
- Integration: Modify replay buffer sampling
- Logging: Track reshaping statistics

Dataset traces:

- Source: Gheibi & Weyns DeltaIoT datasets
- Interference levels: 0.0, 0.2, 0.4, 0.6, 0.8, 1.0
- Message loads: Low, medium, high
- Training/validation split: 70/30 or cross-validation

Hyperparameter choices:

Parameter	Value
Learning rate η	0.0001
Discount factor γ	0.99
Exploration rate ϵ	0.1 (decayed)
Reshaping probability ρ	0.2
Replay buffer size	100,000
Minibatch size	64
Target update frequency	1,000 steps
Network architecture	[128, 128]

Table 2: RS-DRL hyperparameter settings.

4.12 Chapter summary and transition to evaluation (Chapter 5)

This chapter presented the RS-DRL architecture and method:

Key contributions:

- Nine-stage adaptation cycle integrating RS-DRL with MAPE-K
- Temporal separation of runtime adaptation and asynchronous learning
- Randomized reward reshaping algorithm for accelerated learning
- Knowledge-based coordination of heterogeneous processes
- Practical implementation using Gymnasium and Stable-Baselines3

Architectural principles:

- Process-oriented organization for traceability
- Verification as justification for learning-based decisions
- Asynchronous learning for continuous operation
- Modular design enabling independent component evolution

Transition to evaluation:

Chapter 5 evaluates RS-DRL through comprehensive experiments on DeltaIoT:

- Learning performance: Convergence speed, sample efficiency
- Generalization: Cross-environment policy transfer
- Comparison: Baseline DQN, HER, search-based methods
- Ablation studies: Impact of reshaping probability ρ
- Scalability: Performance across adaptation space sizes